- Perform regular health checks and troubleshoot the apps with auto placement, auto restart, auto replication and auto scaling.

Kubernetes architecture

Kubernetes follows the master-slave architectural model.

The following are the components of Kubernetes' architecture.

- **Kubernetes master:** This is the main controlling unit of the cluster, which performs the tasks of managing all sorts of workloads and communication across the entire system. It controls all nodes connected in the cluster.
- **API server:** This is regarded as a key component to serve the Kubernetes API using JSON over HTTP, and provides both internal as well as external interfaces to Kubernetes.
- **Scheduler:** This tracks the resource utilisation of every node to ensure that no excess workload is scheduled with regard to available resources. The scheduler keeps track of resource requirements, availability, and a variety of user-provided constraints and policy directives like QoS, location of data, etc.
- **Replication controller:** This controls how many identical copies of a pod should be running on the cluster.
- **Nodes:** These are machines that perform the requested and assigned tasks. The rubbernecks master controls all nodes connected in the cluster.
- **Kubelet:** This is primarily responsible for knowing the state of each node with regard to its performance. It performs the tasks of starting, stopping and maintaining application containers as organised by the control plane.
- **Proxy:** This is regarded as the implementation of the network proxy and load balancer, and provides abstraction of the service level along with other network operations. It is also responsible for routing traffic to the appropriate container, based on the IP and port number of the incoming request.
- **Pod:** This is a group of one or more containers deployed in a single node. All containers in a pod share an IP address, IPC, host name and other resources. Pods abstract network and storage away from the underlying container, which helps in making containers move across the cluster in a proper manner.

Amazon ECS (Elastic Container Service)

Amazon Elastic Container Service (Amazon ECS) is a highly scalable and fast container management service provided by Amazon AWS (Amazon Web Services). It supports Docker containers and allows users to run applications on a cluster of Amazon EC2 instances, eliminating the need for end users to install, operate and scale their own cluster management infrastructure.

It allows users to launch and stop container-based applications with simple API calls, and to create state-of-art clusters via a centralised service. Amazon ECS also helps users to schedule the placement of containers across the cluster depending on resource needs, isolation policies and availability requirements.

Amazon ECS can be used to create a consistent deployment and build experience. It can manage and scale batch and extract-transform-load (ETL) workloads, and build sophisticated application architectures on a microservices model. Its main features are listed below.

- **Task definition:** The task definition is a text file, in JSON format. It describes the containers that together form an application. Task definitions specify various parameters for the application, e.g., container image repositories, ports, storage, etc.
- **Tasks and the scheduler:** A task is an instance of a task definition, created at runtime on a container instance within the cluster. The task scheduler is responsible for placing tasks on container instances.
- **Service:** A service is a group of tasks that is created and maintained as instances of a task definition. The scheduler maintains the desired count of tasks in the service. A service can optionally run behind a load balancer, which distributes traffic across the tasks that are associated with the service.
- **Cluster:** A cluster is a logical grouping of EC2 instances on which ECS tasks are run.
- **Container agent:** The container agent runs on each EC2 instance within an ECS cluster. The agent sends telemetry data about the instance's tasks and resource utilisation to Amazon ECS. It will also start and stop tasks based on requests from ECS.
- **Image repository:** Amazon ECS downloads container images from container registries, which may exist within or outside of AWS, such as an accessible private Docker registry or Docker hub.

Figure 2 highlights the architecture of Amazon ECS. The AWS services that are used with ECS are:

- Elastic load balancer: Routes traffic to containers.
- Elastic block store: Provides persistent block storage for ECS tasks.
- CloudWatch: Collects metrics from ECS.
- Virtual private cloud: ECS cluster runs within the virtual private cloud.

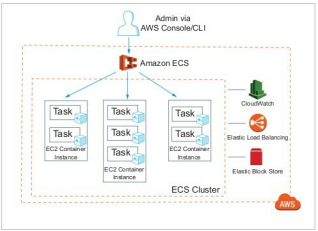

Figure 2: Amazon ECS architecture and components